Angewandte KI & KI-Engineering (AKE)

Block B – Spezialisierte Anwendungsgebiete

Selbststudium Woche 4: Sprachmodelle anpassen

Wintersemester 2026/27

● Organisatorisches – Selbststudium Woche 4	
Termin	Do 29.10.2026 (KW 44): keine Präsenz. Sie arbeiten den Lernpfad selbstständig und in Ihrem eigenen Tempo durch
Umfang	etwa 4,5 Stunden = das Zeitfenster der Sitzung; jeder Abschnitt braucht etwa 20–30 Minuten
Material	Skript *Moderne NLP & LLM-Anpassung* (PDF) · Skript *Sequenzmodelle & Transformer* · Kursrepository: `lab03/`, `lab04/` · die Links in diesem Lernpfad
Vorab	**Mit Internet:** `git pull`, `uv sync`, `lab04/setup_check.ipynb` einmal ausführen (lädt SmolLM2-135M-Instruct, ≈ 270 MB). Danach läuft alles offline
Testat	Quiz „Testat Woche 4“ in Teams unter „Aufgaben“, Pflicht, unbenotet: bestanden ab 60 %. Das Testat erhebt Ihren Lernstand; die Fragen folgen den Leitfragen und den grauen Kästen „Für das Testat notieren“

So arbeiten Sie mit diesem Lernpfad

Die Woche hat eine Leitfrage: *Wie passt man ein Sprachmodell an eine eigene Aufgabe an – und woran erkennt man, dass es geklappt hat?* Sie arbeiten in zwei Teilen: **Teil A** – sieben Stationen in der Reihenfolge des Skripts, in denen Sie sich die Themen mit Skript und externen Quellen erarbeiten. **Teil B** – das Lab am Stück, in dem Sie das Gelernte im Notebook anwenden. Beginnen Sie Teil B erst, wenn Teil A abgeschlossen ist.

1. **Lernziel lesen.** Was soll ich am Ende können?
2. **Leitfragen lesen** und stichpunktartig beantworten, während Sie lesen. Wer alle Leitfragen beantworten kann, besteht das Testat.
3. **Quellen in dieser Reihenfolge:** (1) Skript – Pflicht, (2) Zweitquelle – Pflicht, kurz und gezielt, (3) Vertiefung – optional. Die Schlüsselwörter sind der Einstieg für eigene Recherche.
4. **Notieren** Sie, was im grauen Kasten steht – Sie brauchen es im Testat.
5. **Weiter**, wenn Sie das Kriterium am Ende der Station erfüllen.

Teil	Abschnitt
A	S0 Destillation und Modellgröße (Rest Woche 3)
A	S1 Vom Sprachmodell zum Werkzeug, Decoder
A	S2 Prompting, RAG oder Fine-Tuning?
A	S3 Instruction-Tuning
A	S4 LoRA
A	S5 Präferenzoptimierung: RLHF und DPO
A	S6 Evaluation
B	L1 Lab 03 A5 · L2 Lab 04 A1–A3 · L3 A4–A5 · L4 A6; Selbsttest in den Rechenpausen
–	Testat als Quiz in Teams (Aufgaben)

Teil A: Themen erarbeiten

S0 · Destillation und Modellgröße

Lernziel: Wissensdestillation erklären; die Modellgröße als Entwurfsentscheidung bewerten

Leitfragen

1. Warum enthält das weiche Label eines Lehrermodells mehr Information als das harte Label?
2. Was bewirkt die Temperatur T in der Softmax, und wozu dient der Faktor T^2 im Destillationsverlust?
3. Was hat DistilBERT gegenüber BERT-base eingespart, und was hat es behalten?
4. Warum bringt ein größeres Modell immer weniger zusätzliche Genauigkeit, und nach welchen fünf Kriterien wählt man die Modellgröße?

Schlüsselwörter für die Recherche: Wissensdestillation / Knowledge Distillation · weiche Ziele / Soft Targets · Temperatur / Temperature Scaling · Lehrer–Schüler / Teacher–Student · DistilBERT · abnehmender Ertrag / Diminishing Returns · Scaling Laws

Quellen

- **Pflicht:** Skript *Sequenzmodelle & Transformer*, Abschn. 7.5–7.6 (S. 25–26)
- **Zweitquelle** – das Skript fasst die Destillation nur kurz, deshalb drei Teile:
  - V. Sanh, „Smaller, faster, cheaper, lighter: Introducing DistilBERT“ – Abschnitte „Knowledge Distillation“ bis „Model performances“
  - Intel Labs Distiller, „Knowledge Distillation“ – Softmax mit Temperatur und kombinierter Verlust
  - G. Hinton et al. (2015), „Distilling the Knowledge in a Neural Network“, Abschn. 2, letzter Absatz – warum T^2
- **Vertiefung:**
  - Hinton et al. (2015), https://arxiv.org/pdf/1503.02531 Abschn. 1 (Beispiel BMW, Müllwagen, Karotte)
  - V. Sanh et al. (2019), DistilBERT, Abschn. 2–3 und Tab. 1
  - J. Kaplan et al. (2020), „Scaling Laws for Neural Language Models“, Abschn. 1.1 und Abb. 1

Selbst rechnen

Lehrer-Logits (3; 2; 1; −1) für „bellt“, „knurrt“, „schläft“, „Auto“: Softmax bei T = 1 und T = 2. Kontrollwerte: (0,66; 0,24; 0,09; 0,01) und (0,47; 0,29; 0,17; 0,06).

Für das Testat notieren
ein eigenes Temperatur-Beispiel: Logits (2; 1; 0) bei T = 2 · was die Destillation überträgt und was DistilBERT eingespart hat

Weiter, wenn: Sie können das Temperatur-Beispiel ohne Vorlage nachrechnen und erklären, warum „Auto“ bei T = 2 mehr Gewicht bekommt – und warum die Reihenfolge trotzdem gleich bleibt.

S1 · Vom Sprachmodell zum Werkzeug – Decoder

Lernziel: erklären, wie ein Decoder-Sprachmodell Text erzeugt; Perplexität und Dekodierstrategien unterscheiden

Leitfragen

1. Was unterscheidet ein Basismodell von einem Assistenzmodell, und welche Trainingsphasen liegen dazwischen?
2. Wie erzeugt ein Decoder einen Text Token für Token, und was ist sein Trainingsziel?
3. Was misst Perplexität, und wie hängt sie mit dem mittleren Cross-Entropy-Verlust zusammen?
4. Wie wird aus Logits ein Token (greedy, Temperatur, Top-p) – und warum ist Sampling für eine JSON-Extraktion ungeeignet?
5. Was leistet Few-Shot-Prompting, und wo endet es?

Schlüsselwörter für die Recherche: autoregressives Sprachmodell / Autoregressive Language Model · Next-Token Prediction · Perplexität / Perplexity · Dekodierung / Decoding: Greedy, Temperature, Top-k, Top-p (Nucleus Sampling) · Basismodell vs. Instruct-Modell · Few-Shot-Prompting

Quellen

- **Pflicht:** Skript *Moderne NLP & LLM-Anpassung*, Abschn. 1 (S. 6–7, Basismodell und Assistent, Trainingsphasen, Leitbeispiel Störmeldungen) sowie 2.1–2.3 (S. 8–10, Faktorisierung, Perplexität, Dekodierung, Prompting und In-Context-Learning)
- **Zweitquelle:**
  - P. von Platen, „How to generate text“ (Hugging Face Blog) – „Introduction“, „Greedy Search“, „Sampling“, „Top-K“, „Top-p“; *Beam search überspringen*
  - Hugging Face Docs, „Perplexity of fixed-length models“, nur die Einleitung
- **Vertiefung:**
  - 3Blue1Brown, „But what is a GPT?“ (Textfassung; Video ca. 27 Min.)
  - Bishop & Bishop (2024), *Deep Learning*, Abschn. 12.3.1–12.3.2 (SpringerLink)
  - A. Holtzman et al. (2020), „The Curious Case of Neural Text Degeneration“, Abschn. 3

Für das Testat notieren
warum für eine Extraktion greedy dekodiert wird · selbst rechnen: Perplexität bei mittlerem Verlust 2,0 · wie viele Token bei Top-p 0,9 und p = (0,5; 0,3; 0,15; 0,05) im Topf bleiben

Weiter, wenn: Sie können aus einem mittleren Verlust die Perplexität berechnen und greedy, Top-k und Top-p an einem Beispiel unterscheiden.

S2 · Prompting, RAG oder Fine-Tuning?

Lernziel: Fine-Tuning und RAG abgrenzen

Leitfragen

1. Was ist der Unterschied zwischen *Wissen* und *Verhalten*, und welches Mittel adressiert was?
2. Wie läuft eine Anfrage durch ein RAG-System: Was ist ein Chunk, was macht der Retriever, was speichert die Vektordatenbank, was steht am Ende im Prompt?
3. Woran scheitert RAG typischerweise?
4. Wann ist ein Werkzeugaufruf besser als RAG und Fine-Tuning?

Schlüsselwörter für die Recherche: Retrieval-Augmented Generation (RAG) · Chunking · Retriever: dicht (Embeddings) und lexikalisch (BM25) · Vektordatenbank / Vector Database · Fine-Tuning vs. RAG · Werkzeugaufruf / Tool Use, Function Calling

Quellen

- **Pflicht:** Skript, Abschn. 3 (S. 10–12, Tabelle „Vier Mittel im Vergleich“ und Entscheidungsweg)
- **Zweitquelle:**
  - OpenAI, „Optimizing LLM Accuracy“ – bis einschließlich „Understanding the tools“
  - Anthropic, „Introducing Contextual Retrieval“ – Abschnitte „A primer on RAG“ und „The context conundrum in traditional RAG“
- **Vertiefung:**
  - P. Lewis et al. (2020), „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks“, Abschn. 1–2
  - Pinecone, „Retrieval-Augmented Generation“
  - Alammar & Grootendorst (2024), *Hands-On Large Language Models*, Kap. 8 (O’Reilly)

Für das Testat notieren
die vier Anwendungsfälle aus Beispiel „Vier Anwendungsfälle zuordnen“ und warum sie so zugeordnet werden · was in einem RAG-System wo gespeichert wird

Weiter, wenn: Sie können in einem Satz sagen, was RAG ändert und was Fine-Tuning ändert.

S3 · Instruction-Tuning

Lernziel: erklären, wie ein Instruction-Tuning-Beispiel aufgebaut ist und warum der Verlust nur auf der Antwort berechnet wird

Leitfragen

1. Wie sieht ein Trainingsbeispiel im Chat-Format (ChatML) aus?
2. Warum wird der Verlust nur auf den Antwort-Token berechnet?
3. Was zeigen InstructGPT (1,3 Mrd. Parameter schlagen 175 Mrd.) und FLAN – und warum widersprechen sich ihre Befunde zu kleinen Modellen nicht?
4. Warum schlägt Datenqualität die Datenmenge?
5. Was ist katastrophales Vergessen, und was ist die Alignment-Steuer?

Schlüsselwörter für die Recherche: Instruction-Tuning · Supervised Fine-Tuning (SFT) · Chat-Template / ChatML · Loss-Maske / Loss Masking (−100) · InstructGPT · FLAN · LIMA

Quellen

- **Pflicht:** Skript, Abschn. 4.1–4.4 (S. 12–14)
- **Zweitquelle:**
  - Hugging Face LLM Course, Kap. 11.2 „Chat Templates“ – „Base Models vs Instruct Models“ und „Common Template Formats“
  - S. Raschka, „When to mask prompt tokens“
- **Vertiefung:**
  - OpenAI, „Aligning language models to follow instructions“
  - Google Research, „Introducing FLAN“
  - C. Zhou et al. (2023), LIMA, Abschn. 1 und 5
  - Hands-On LLMs, https://github.com/handsOnLLM/Hands-On-Large-Language-Models Kap. 12, „Supervised Fine-Tuning“

Für das Testat notieren
was eine Loss-Maske bewirkt · was InstructGPT und FLAN zeigen · warum Datenqualität vor Datenmenge geht

Weiter, wenn: Sie können sagen, was das Modell lernte, wenn der in allen 300 Beispielen gleiche Systemprompt nicht maskiert wäre.

S4 · LoRA – parameter-effizientes Fine-Tuning

Lernziel: die LoRA-Idee erklären; trainierbare Parameter und Speicherbedarf berechnen

Leitfragen

1. Wie viel Speicher braucht volles Fine-Tuning je Parameter, und wofür?
2. Was ist die LoRA-Idee: Was bleibt eingefroren, was wird trainiert, was bedeuten r und α?
3. Wie viele trainierbare Parameter hat LoRA für eine Matrix der Form d × k?
4. Welche Matrizen passt man an, und welcher Rang genügt?
5. Was macht QLoRA zusätzlich, und warum kostet ein eingerechneter Adapter keine Zusatzlatenz?

Schlüsselwörter für die Recherche: Parameter-effizientes Fine-Tuning (PEFT) · LoRA / Low-Rank Adaptation · Adapter, Prompt-Tuning · Rang r, `lora_alpha` · `target_modules` (`q_proj`, `v_proj`) · QLoRA, NF4 · Adapter einrechnen / merge

Quellen

- **Pflicht:** Skript, Abschn. 5.1–5.3 und 5.6–5.8 (S. 14–20)
- **Zweitquelle:**
  - Hugging Face LLM Course, Kap. 11.4 „LoRA“ – „Understanding LoRA“ bis „Merging LoRA Adapters“
  - Hugging Face Docs, „Model training anatomy“, Abschnitt „Tensors“.

Achtung: Die Doku rechnet mit 18 Byte je Parameter (Gradienten in fp32), das Skript mit 16 Byte (Gradienten in 16 Bit). Beides ist richtig – es kommt auf die Annahme an.

- **Vertiefung:**
  - E. Hu et al. (2022), LoRA, Abschn. 4
  - T. Dettmers et al. (2023), QLoRA, Abschn. 3
  - Hugging Face Blog, „Making LLMs even more accessible with bitsandbytes, 4-bit quantization and QLoRA“
  - Hands-On LLMs, Kap. 12, „Instruction Tuning with QLoRA“

Für das Testat notieren
LoRA-Parameter für eine 1 024 × 1 024-Matrix mit r = 4 · Speicher für volles Fine-Tuning mit 16 Byte je Parameter · welche Matrizen man mit welchem Rang anpasst · was QLoRA anders macht

Weiter, wenn: Sie können ohne Skript erklären, warum B mit 0 startet und warum ein Adapter nur wenige MB groß ist.

S5 · Präferenzoptimierung: RLHF und DPO

Lernziel: Methoden der Präferenzoptimierung nennen und unterscheiden; den DPO-Verlust berechnen

Leitfragen

1. Warum nutzt man Präferenzen statt weiterer Demonstrationen?
2. Welche drei Schritte hat RLHF, und welche Daten braucht jeder?
3. Was sagt das Bradley-Terry-Modell, und warum zählt nur die Differenz der Scores?
4. Was ist die Kernidee von DPO, was fällt gegenüber RLHF weg, und wie viele Modelle liegen im Speicher?
5. Was bewirkt β, und wessen Präferenzen lernt das Modell?

Schlüsselwörter für die Recherche: Präferenzoptimierung / Preference Optimization · RLHF / Reinforcement Learning from Human Feedback · Belohnungsmodell / Reward Model · Bradley-Terry · PPO · KL-Strafe / KL Penalty · DPO / Direct Preference Optimization · β · Alignment

Quellen

- **Pflicht:** Skript, Abschn. 6 komplett (S. 20–23), mit beiden Rechenbeispielen
- **Zweitquelle:**
  - N. Lambert et al., „Illustrating Reinforcement Learning from Human Feedback (RLHF)“ – Abschnitte „Reward model training“ und „Fine-tuning with RL“
  - K. Rasul et al., „Fine-tune Llama 2 with DPO“ – Abschnitt „DPO vs PPO“
- **Vertiefung:**
  - N. Lambert, *RLHF Book*, Kap. 8 „Direct Alignment Algorithms“, Abschnitt „How DPO Works“
  - L. Ouyang et al. (2022), InstructGPT, Abschn. 3.2 (Daten je Schritt) und 5.2 (wessen Präferenzen)
  - R. Rafailov et al. (2023), DPO, Abschn. 4
  - Hands-On LLMs, Kap. 12, „Preference Tuning with DPO“

Für das Testat notieren
Bradley-Terry-Verlust für zwei Scores selbst rechnen (wie im Beispiel des Skripts) · DPO-Verlust für eigene Log-Wahrscheinlichkeiten · Zahl der Modelle im Speicher bei DPO und bei RLHF mit PPO · Bedeutung von β

Weiter, wenn: Sie können erklären, warum DPO ohne Belohnungsmodell und ohne Reinforcement Learning auskommt.

S6 · Evaluation: Benchmarks lesen, Eval-Setup entwerfen

Lernziel: einen LLM-Benchmark kritisch bewerten (Bewerten); Eval-Ergebnisse mit Intervall und gepaartem Vergleich analysieren (Analysieren); die Bausteine eines Eval-Setups benennen

Leitfragen

1. Warum steht das Eval-Setup vor dem ersten Training?
2. Was messen MMLU-Pro und IFEval, und warum liefern zwei Quellen für dasselbe Modell verschiedene Werte?
3. Wie groß ist die Unsicherheit eines Ergebnisses auf 50 oder 100 Beispielen, und wie vergleicht man zwei Modelle gepaart?
4. Wie funktioniert LLM-as-a-Judge, welche Verzerrungen sind bekannt, und wie begegnet man ihnen?

Schlüsselwörter für die Recherche: Evaluation von Sprachmodellen · Benchmark, MMLU-Pro, IFEval · Messprotokoll / Evaluation Protocol (Few-Shot, Multiple-Choice-Format) · Kontamination / Contamination · Standardfehler, Konfidenzintervall / Confidence Interval · Vorzeichentest / Sign Test · LLM-as-a-Judge · Position Bias, Verbosity Bias, Self-Enhancement Bias

Quellen

- **Pflicht:** Skript, Abschn. 7.1–7.3, 7.5 und 7.6 (S. 23–27) sowie 8.1–8.2 (S. 27–29, Eval-Setup in sieben Schritten, Eval-Karte, Fehlerkategorien)
- **Zweitquelle:**
  - C. Fourrier et al., „What's going on with the Open LLM Leaderboard?" – Abschnitte „1001 flavors of MMLU" und „MMLU comes in all shapes and sizes"
  - L. Zheng et al. (2023), „Judging LLM-as-a-Judge", Abschn. 3.3 (Verzerrungen)
- **Vertiefung:**
  - E. Miller (2024), „Adding Error Bars to Evals", Abschn. 2.1 und 4.2
  - Y. Wang et al. (2024), MMLU-Pro, Abschn. 3 und 6.3
  - E. Yan, „Evaluating the Effectiveness of LLM-Evaluators"
  - Zheng et al., Abschn. 3.4 (Gegenmittel)
  - Hands-On LLMs, Kap. 12, „Evaluating Generative Models"

Für das Testat notieren
einen Vorzeichentest und ein 95-%-Intervall selbst rechnen · warum Benchmark-Werte vom Messprotokoll abhängen · drei Judge-Verzerrungen

Weiter, wenn: Sie können ein Eval-Ergebnis so berichten, dass es in eine Eval-Karte passt: Wert, n, Intervall, gepaarter Vergleich.

Teil B: Lab

Jetzt wenden Sie an, was Sie in Teil A erarbeitet haben. Arbeiten Sie L1–L4 am Stück; jede Lab-Station nennt die Station aus Teil A, auf die sie zurückgreift.

Hinweise zum Notebook

Ausführen-Modus. Einige Teilaufgaben sind hier nicht Pflicht, ihre Zellen müssen aber laufen, weil spätere Aufgaben darauf aufbauen (`auswerten` aus 2.5 wird in Aufgabe 6 gebraucht, `kodiere` aus 3.2 in Aufgabe 4). Im Ausführen-Modus übernehmen Sie die Lücken direkt aus `lab04_loesung.ipynb` und führen die Zellen aus – ohne die Leitfragen dazu. **Fallback:** Liegt die Hochrechnung aus `setup_check` über 20 Min. oder dauert ein Trainingsschritt länger als 8 s, setzen Sie in Aufgabe 4 `NUTZE_REFERENZ_ADAPTER = True`. Alle weiteren Aufgaben laufen damit unverändert.

L1 · Lab 03, Aufgabe 5 – DistilBERT feinjustieren

Lernziel: einen Text-Classifier feinjustieren und fair mit eingefrorenen Embeddings vergleichen · Bezug: S0

Ihr Notebook aus Woche 3 öffnen und alle Zellen bis einschließlich Aufgabe 4 erneut ausführen (Embeddings kommen aus dem Cache), dann 5.1–5.3: Kopf anlegen, AdamW mit Warmup, Training starten (125 Schritte, ≈ 1,5–3 Min., ältere Laptops bis 6 Min.). **Kontrollwerte:** Verlust vor dem Training ≈ 0,69 · Testgenauigkeit ≈ 82 %. **Notbremse:** mehr als 3 s je Schritt → `FREEZE = 3` oder `lade_referenz("a5_finetune")`.

Für das Testat notieren

Zahl der trainierbaren Parameter von DistilBERT-FT (Ausgabe in Aufgabe 5) · warum der Verlust vor dem Training bei ≈ 0,69 liegt

L2 · Lab 04, Aufgaben 1–3 – Modell, Decoder, Loss-Maske

Lernziel: den Bauplan eines Decoders, Perplexität und Dekodierung im Code nachvollziehen; die Loss-Maske prüfen · Bezug: S1, S3

`lab04_starter.ipynb` als `lab04_<nachname>.ipynb` kopieren, Setup-Zelle ausführen. Das Notebook bleibt bis L4 geöffnet.

- **A1 (1.1–1.3):** Bauplan lesen, Parameter je Schicht zusammensetzen. Kontrollwert 3 540 096 je Schicht, 134 515 008 gesamt.
- **A2.1–2.2:** Logits und Softmax; Perplexität des Beispiels in 2.2 (p = 0,5; 0,25; 0,8; 0,1). Kontrollwert 3,16.
- **A2.3–2.6 im Ausführen-Modus.** 2.6 erzeugt Antworten auf 50 Störmeldungen (≈ 6 Min. Rechenzeit) – **Rechenpause: Selbsttest, Aufgaben 1–3.** Kontrollwerte: zero-shot 0 % gültiges JSON; few-shot 94 % Format, 78 % Schema, 22 % komplett.
- **A3 im Ausführen-Modus:** 3.0–3.3 mit den Zeilen aus der Lösung ausführen und die Ausgaben lesen. Kontrollwerte: bei `train[0]` 100 Positionen maskiert, 58 Lernziel; Verlust 2,03 mit Maske, 3,53 ohne.

Für das Testat notieren

Anteil der Embedding-Matrix an allen Parametern von SmolLM2 (aus der Ausgabe von A1) · warum der Verlust ohne Maske höher ist

L3 · Lab 04, Aufgaben 4–5 – LoRA trainieren und von innen ansehen

Lernziel: ein LoRA-Training durchführen und die trainierbaren Parameter vorab berechnen · Bezug: S4

- **A4.1:** Zielmodule `q_proj` und `v_proj` ergänzen. Kontrollwert 460 800 trainierbare Parameter (0,34 %). **A4.2:** Training starten (150 Schritte, r = 8, α = 16; je nach Rechner 4–14 Min.) – **Rechenpause: Selbsttest, Aufgaben 4–6.** Kontrollwert: Validierungsverlust 1,82 vorher, 0,53 nachher.
- **A5.1–5.4** nach dem Training: Rang einer Matrix, Rang-1-Update, eigene `LoRALinear`-Klasse, LoRA-Parameter von SmolLM2 von Hand. Kontrollwert 460 800.

Für das Testat notieren
LoRA-Parameter je Schicht von SmolLM2 (aus 5.4)

L4 · Lab 04, Aufgabe 6 – Evaluation vorher und nachher

Lernziel: Eval-Ergebnisse gestuft, mit Intervall und gepaartem Vergleich berichten · Bezug: S6

Dieselben 50 Meldungen mit Adapter bewerten; `vorzeichentest(b, c)` ergänzen (Kontrollwert b = 12, c = 3 ⇒ p ≈ 0,035). Kontrollwerte: 100 % Format, 88 % Schema, 56 % komplett (95-%-Intervall 42–70 %); gepaart gegen few-shot 21 besser, 4 schlechter, p ≈ 0,001.

Für das Testat notieren
Halbbreite des 95-%-Intervalls für 56 %, wenn nur 10 statt 50 Meldungen ausgewertet würden

Weiter, wenn: Sie können Ihr Lab-Ergebnis in einem Satz wie in einer Eval-Karte berichten: Wert, n, Intervall, gepaarter Vergleich.

Selbsttest zu Teil A – in den Rechenpausen von L2 und L3

Rechnen und beantworten Sie ohne Unterlagen, während das Notebook rechnet. Wo Sie hängen bleiben, gehen Sie zurück zur Station.

1. Ein Modell weist den Token einer Antwort die Wahrscheinlichkeiten 0,9; 0,5; 0,2 zu. Berechnen Sie den mittleren Verlust und die Perplexität. (S1)
2. Ein Unternehmen will, dass ein Sprachmodell Fragen zu 3 000 internen Arbeitsanweisungen beantwortet, die quartalsweise überarbeitet werden. Fine-Tuning oder RAG? (S2)
3. Warum werden beim Instruction-Tuning die Prompt-Token mit −100 maskiert, das Ende-Token aber nicht? (S3)
4. Berechnen Sie die LoRA-Parameter für $W_0 \in \mathbb{R}$^(4096×4096) und r = 16 sowie den Anteil an der vollen Matrix. (S4)
5. Zeigen Sie, dass im Bradley-Terry-Modell eine Verschiebung beider Belohnungen um eine Konstante die Vorhersage nicht ändert. (S5)
6. Ihr LoRA-Modell erreicht auf 50 Beispielen 82 %, die Few-Shot-Baseline 74 %; 7 Beispiele wurden besser, 3 schlechter. Ist die Verbesserung gesichert? (S6)

Kür: Nachbereitung

Optional, nicht Teil des Testats:

- **Woche 3:** Skript *Sequenzmodelle & Transformer*, Abschn. 8 (Analyse von Embeddings, Attention und Fehlern); Lab 03 Aufgabe 6 und O1
- **Skript Woche 4:** Abschn. 5.4–5.5 (LoRA-Gradienten, LoRA als Verallgemeinerung), 7.4 (Menschliche Bewertung), 8.3 (Diagnose von LoRA-Trainingsläufen)
- **Lab 04:** Aufgabe 7 (Regressions-Set), O1 und O2; die Leitfragen zu A2.3–2.6 und A3
- **Literatur:** alle Vertiefungsquellen

● Organisatorisches – Selbststudium Woche 4 – Abgabe	
Testat	Quiz „Testat Woche 4“ in Teams (Aufgaben) bis **Mi 04.11.2026, 23:59** abgeben; bestanden ab 60 %